32x32 Color and Grayscale Screen + Converter and writer

Explanations:

Usage:

Other:

Intro:

Yep, it's a 32x32 color display. No mods required (no complexity mod) the rest is just specific stuff. If you want me to make your image, just send it to me. Preferred if it's already 32x32 but if not that's ok too.

In game display:

As the Title says I's a in game 32x32 pixel color and grayscale display. Meaning it has 1,024 pixels, all of which can be set to any value from -1 to 1 with an accuracy of the thousandth place, that being it can have a color such as 0.99 or -0.39 but not 0.851. This display does not need to be written in a certain order, write to any pixel anywhere without affecter another pixel. The display is made of 4 different creations each with 256 pixels (8x32) linked by ankers with springs to hold them together very tightly. Why 4 segments? 1 less laggy 2 NO complexity mod needed for the display. This can run on any server assuming you don't get knocked over, which is unlikely to say the least. This also means it has a lot less chance of crashing your game. Lastly, there are 2 strips of lights on the horizontal and vertical to show where your x and y are.

Display Segment specifics:

First segment has a strip of lights on the left for what x you are at bringing comp to 690 with 668 blocks. Segments 2 and 3 both have comp of 646 at 620blocks and the last segment 634comp at 612blocks. The added comp from the block total is because the segments sit on trends so you can connect them. Also, with the tire pressure set to max it makes sure the displays are the same height (on a flat surface). They all have stabilizers, so they don't fall over or tilt unexpectedly. Each segment has a seat and while driving can turn on a dime, drive forward, or backwards (excluding segment 1 which cannot drive). Each segment has a built-in timer from 0min to 25min. Witch will be explained later. To connect them simply drive them into the last segment so 2 into 1 3 into 2 and 4 into 3. Then press q to link them.

RC:

This creation uses 4 channels in total to operate. The current X position, Current Y position, current color, and write signal. The display revives all the values to write to the respective pixel with given data

25min limit:

As said above each segment has a 25min timer. This I because after a good amount of testing I realized that if a player Is not in a creations seat for 25min or over will simply disappear. Douse not matter if RC is sending data to it or not. Nor if it's anchored to something. So, there's 3 ways to fix that get some other players to sit in the seats for a long period of time, get 4 alts (I'm broke), Or just sit in the seats for 1s every so often. So, the timers when at 22+min will light a red light telling me to go and sit in the seat. Then I press P to reset the timer. Each Segments has its own timer since they are different creations. Hope this helps if you are making muti creation RC builds.

Manual Writer:

In one of the images above you can see a small, tilted panel with a color gradient on it. That is the manual writer. Using it you can manually select any color and write to the display anywhere. It includes a color picker as in it gives you a gradient of colors in order by group there is 24 groups in total each witch have 9* colors. 12 for hue 12 for grayscale. I say 9* because each new group adds 5 new colors because it shows the top 4 from the last group so you can tell where the color gradient is coming from and going. Total of 60 hue and 60 grayscale colors. The controls are as follows:

W – color up 1 step

S – color down 1 step

E – color up 1 group

Q - color down 1 group

Arrow keys – change X and Y position

Space – write color as selected at selected position

Image converter (Python):

This Program was made by **CRITICALS** so BIG thanks to them for this program! This program is made to convert computer images including png, jpg, bmp, webp to hue and gray scale values. As I did not make it, I don't know exactly how it works however it allows the user to import an image, see a preview of what the image will look like in game and adjust many values. For converting to hue/grayscale image Brightness Threshold, and Saturation threshold. As well as for the preview Panel Saturation, and Panel brightness. And of course, resize it. To use it import a image adjust sliders until it have the desired output Then press the Write to game button this will have some text in the console appear were you can write to the game from there.

Writing system (Python):

This is the part that took the most time. When this program is given a list of hue values in the form of -1 to 1 it will control my keyboard to write all the colors in the right position. Paired with the image converter you can take any image and convert it into the in-game display. The hard part is that the program can't see the game, it's kind of typing blind. So, the program stores a bunch of values that is thinks it's at like its own x y and color. Using the specialized creation seen in the video above it can write to the game. You can see it working on the video. That's the program writing to the game. Why you may ask douse it look like the game is in light speed. Well, that image took 20min … yep for 1 image it's 20min. well that's not entirely true. You see that was a relatively slow image, the fastest image you can do is full 1 color. That is because the program does not need to change color. Since it writes left to right if 2 of the same color are in a row it's going to go faster than if it's the same or similar color. Since if they are similar the program does not need to change the color number as much. I'm not going to go into much more detail than that because it won't be helpful to most people but if you have questions let me know. Usage is pretty simple though. After copy and pasting the list from the converter into the "siganls.txt" file just press run it will tell you the rest. Press and hold "Z" on your keyboard at any time to pause the writing process adjust values check key bindings reset the segment timer's ext.

Using it yourself Without external program:

First, you need the 4 display segments and the Manual Display writer. After that spawn the 4 segments in order 1 2 3 and 4 then connect them using the wedges in the back so the screen is flush. Then with the Manual Display Writer you can start writing on the display. See “Manual Writer” section for more information. Also note when any red lights under the display light up that means you need to pause sit in the segment that has the red light on and press “P” until the timer at the bottom of the display is yellow is 0. Then you can continue writing.

Using it yourself with external program:

This is a bit complicated and will require a windows pc version of trail makers as it needs to run an python file directly (they are open source no copyright), they are on my GitHub however they have not been tested on other operating systems so there is great change it won’t work without modification. This will also require a bit of technical experience. To start finding a file preferably already pixel art but not required. Then make sure the height and width of the image is the same if not resize it so the aspect ratio is 1 to 1. Then open TSR_V1.py file. Adjust the sliders, so the image in the middle look close to the image on the left. (Note if adjusting the panel brightness or saturation you are going to need to change those settings in game on the hue blocks to match). Now in game set up the display with the Program writer instead of the Manual writer as instructed in the Using it yourself without external programs. Then started writing the console (by pressing enter) it will start a countdown of 3s make sure by the time it’s done the focused window is the trailers window it will then start controlling your keyboard and type keys for you to write the image. At any time, press and hold Z it will some up with a help menu giving you many options such as manual writing commands change preserved values current perceived data and more. Once the program is finished writing the game it will automatically close the program. You can close the program at any time without worrying about data corruption or leaking everything is handled during termination.

Last notes:

If you want me to make proper documentation let me know but for now, I’m not planning to because I have no idea how many people are going to use this. The files are on my GitHub including the 4 segments, the manual and program as well as the python program.

Definitions/key:

Pixel:

This refers to the in-game hue-block with an accumulator that stores the color.

X (Position):

The Display is represented as a grid with 1x1 at the top left corner and 32x32 at the bottom right corner. The X is the horizontal plane.

Y (Position):

The Display is represented as a grid with 1x1 at the top left corner and 32x32 at the bottom right corner. The Y is the Vertical plane.

Comp or comp:

The creations complexity

Converter:

This is in reference to the python program that converts a png, jpg, ext. to hue/grayscale values.

Credits:

ImACouch – In game creations and Pixel Writer

CRITICALS – computer image to Hue and Gray scale converter program

Links:

GitHub: https://github.com/ImACouch/Trailmakers

TrailOS YouTube: https://www.youtube.com/@ajfun5595